Greenhouse Management

Technical Details and Technology Selection

by Kaloyan Milev

Table of Contents

1 Purpose....1
2 Controller and Firmware....1
3 Soil Moisture and Reservoir Sensing....2
4 Temperature Humidity and Light....4
5 Watering Hardware and Local Display....5
6 Communication....6
7 Backend and Database....7
8 Dashboard and Hosting....8

1 Purpose

This document explains the main hardware and software choices for the one-plant greenhouse prototype. The comparisons consider reliable local watering, compatibility, development time, learning value and cost within the four-week scope.

The Analysis and Advice document defines the requirements; this document records the technology choices and their rationale. Implementation and test results are covered in the Realisation and Verification and Validation documents. The university provides the ESP32, DHT11 and M5Stack Watering Unit. Linked sources support the comparisons; proposed capabilities are not evidence of successful testing.

2 Controller and Firmware

2.1 Microcontroller

The microcontroller reads the sensors and operates the pump. Local watering control must remain available when the dashboard is unavailable, subject to the sensor checks and safety limits.

Option	Main advantage	Main limitation
ESP32 DevKitC V4 (Available)	Built-in Wi-Fi, analogue sensor inputs and connections for the display. The existing board uses an ESP32-WROOM-32D module.	Uses 3.3 V input signals, so sensor and pump interfaces need voltage checks. Wi-Fi must not block the watering logic.
Arduino Uno R3	Straightforward board for basic sensing and pump control; six analogue inputs.	No built-in network connection. Extra communication hardware adds wiring and integration work.
Raspberry Pi Pico W	Built-in Wi-Fi and support for C/C++ or MicroPython. A practical alternative for connected sensing.	Would require another board and a different software setup. Its three exposed analogue inputs leave little room beyond soil, water and light.

My choice: Use the ESP32 that I already have. Its built-in Wi-Fi and sensor interfaces suit the connected greenhouse without an extra communication module. Pico W could also meet these needs, but offers no clear benefit that would justify buying another board for this prototype.

Sources: ESP32 datasheet | Uno R3 | Pico W

2.2 Programming framework

Option	Main advantage	Main limitation
C with ESP-IDF	Direct access to ESP32 drivers, networking and task scheduling, which fits my embedded C learning goal.	More setup and configuration to learn than a simple Arduino sketch.
Arduino framework for ESP32	Convenient C++ APIs and many examples for quickly connecting modules.	Adds an abstraction layer. Using it as the main framework gives less practice with ESP-IDF APIs.
MicroPython	Interactive Python makes small experiments and sensor checks convenient.	Does not develop my C skills. Memory usage and timing depend on the interpreter as well as the program.

My choice: Use C with ESP-IDF to build on my embedded experience and learn the controller's drivers and networking. Arduino is a sensible fallback if framework setup threatens the schedule. Vim and Git remain the editing and version-control tools, but they do not determine the device's behaviour.

Sources: ESP-IDF | Arduino ESP32 | MicroPython

3 Soil Moisture and Reservoir Sensing

3.1 Soil moisture

Soil moisture determines whether watering is needed. The useful measurement is a repeatable difference between dry and sufficiently wet soil. A calibrated relative percentage is an indicator, not a measurement of volumetric water content.

Option	Main advantage	Main limitation
Capacitive probe in the M5Stack Watering Unit	Already available together with the pump. Capacitive sensing reduces the exposed-electrode corrosion problem of resistive probes.	Its reading still depends on soil, placement and calibration. The electronics need protection from water.
Separate capacitive probe Example: DFRobot SEN0193	The sensor can be positioned and replaced independently of the pump assembly.	Requires another part and separate wiring. It still needs calibration in the actual soil.
Resistive probe	A simple way to detect changes in soil conductivity.	Exposed electrodes can corrode, and dissolved salts affect the reading. This creates extra maintenance and uncertainty.

My choice: Use the capacitive probe in the available M5Stack unit. It avoids buying a second soil sensor and fits the one-plant setup. A separate capacitive probe becomes useful if the combined unit cannot be positioned well or needs independent replacement. Calibrate the selected probe in the actual pot before choosing watering thresholds.

Sources: M5Stack Watering Unit | DFRobot SEN0193

3.2 Reservoir water detection

The core requirement is to detect too little water for the pump. Measuring the exact number of litres would add work without changing that decision.

Option	Main advantage	Main limitation
K-0135 water sensor	A compact sensor for detecting water in a small reservoir. Can support a low-water threshold instead of calculating the full volume.	Electrical compatibility, mounting and reliable detection at the cutoff level need to be checked in the installed setup.
Float switch	Provides a simple on/off signal at a chosen water level. No conversion to a continuous level is needed.	Needs secure mounting and room for the float to move; it can stick or become obstructed.
External capacitive sensor	Can detect water through a suitable non-metal reservoir wall without touching the water.	Must match the wall material and thickness, and needs a compatible power and signal interface.
Ultrasonic sensor	Measures distance to the water surface, allowing a continuous level estimate.	Needs suitable clearance and mounting. Distance-to-volume conversion and surface effects add complexity.

My choice: Use the K-0135 water sensor installed in the prototype for reservoir low-water detection. The project needs a low-water signal rather than an exact volume measurement. Check its readings at the installed cutoff level and use them to set the low-water threshold. A float switch is an alternative if more dependable detection is needed in the chosen container.

Sources: Level-sensing technologies | Non-contact capacitive sensing

4 Temperature Humidity and Light

4.1 Air temperature and humidity

These readings help the caretaker recognise unsuitable growing conditions. The core system reports them and raises alerts; it does not control heating or humidity.

Option	Main advantage	Main limitation
DHT11 Available module	Measures both quantities through one digital signal connection. Suitable as a starting point for basic monitoring.	Limited range and accuracy: the cited guide lists about ±2 °C and ±5 percentage points of relative humidity for its DHT11.
DHT22 / AM2302	The cited guide lists a wider range and temperature accuracy of ±0.5 °C, while retaining a simple digital connection.	Needs a replacement purchase and integration. Readings are limited to about once every two seconds.
BME280 module	Measures temperature and humidity using I²C or SPI. Bosch specifies humidity accuracy of ±3 percentage points under its stated conditions.	Requires a different driver and a suitable breakout board. Its additional pressure measurement is not needed here.

My choice: Start with the available DHT11 because it covers the two required measurements without a new purchase. Retain it only if its readings are useful for the chosen plant's alert limits. For example, a sensor with ±2 °C error cannot reliably distinguish conditions only 1 °C apart. DHT22 or BME280 is an upgrade if closer limits or a wider range are needed. Datasheet accuracy is not proof of accuracy after installation.

Sources: DHT11 and DHT22 comparison | Bosch BME280

4.2 Ambient light

Light monitoring is a Should Have. It helps the caretaker notice changes in illumination; automatic lighting remains outside the scope.

Option	Main advantage	Main limitation
Photoresistor module	Its electrical resistance changes with light. A simple analogue reading can show whether conditions are getting brighter or darker.	Response varies between parts. Readings show relative light levels and cannot be labelled as lux without a supported conversion.
BH1750 module	Provides digital readings in lux, a standard measure of visible illumination, through I²C.	Needs an I²C driver. Lux describes human-visible light and does not directly measure the light used in photosynthesis.

My choice: Use a photoresistor module because relative light changes are enough for this feature. Its analogue output is straightforward to read with the ESP32. Choose a BH1750 if readings in lux become a requirement. The photoresistor's output voltage must fit the ESP32 input, and its readings should be presented as relative light levels.

Sources: Photoresistors | BH1750 module

5 Watering Hardware and Local Display

5.1 Pump and power supply

The pump must move water from the reservoir to the pot. The important factors are delivered water, lifting height, power needs and how easily it can be connected and maintained.

Option	Main advantage	Main limitation
M5Stack Watering Unit Available pump	Combines a pump and soil probe, with a control input and supplied tubing. Reduces separate-part assembly.	Actual water delivery depends on tubing and height. The combined arrangement may restrict positioning.
Separate small DC pump	Allows the pump and soil sensor to be selected and mounted independently.	Needs a compatible switching circuit, supply and tubing; selecting and assembling them adds work.
Peristaltic pump	Moves liquid by squeezing tubing, keeping water out of the pump mechanism. Useful when controlled dosing matters.	Needs suitable replaceable tubing and its own drive circuit. Delivery per second still needs calibration.

My choice: Use the available M5Stack pump and tubing for the single pot. A separate or peristaltic pump is justified if the existing unit cannot deliver the required amount at the installed height. Limit each watering run rather than assuming that running the pump for longer is harmless.

Choose a regulated 5 V, 2.5 A pump supply. The specified 5 W at 5 V implies about 1 A; startup demand still needs checking. Power the ESP32 through USB and share signal ground with the pump unit. Keep the external pump supply separate from the board's USB 5 V rail; reconcile this with the schematic's shared 5 V net before wiring. GPIO controls PUMP_EN, not motor power.

Sources: M5Stack pump and pin map | Peristaltic pump example

5.2 Local display

A screen at the plant lets the caretaker check readings and warnings without opening the dashboard.

Option	Main advantage	Main limitation
0.96 inch SSD1306 OLED 128 × 64 pixels	Displays text and simple symbols in a compact module. Flexible pixel layout allows readings and warnings to share the screen.	Small screen area limits text size and how much information fits at once.
16 × 2 character LCD	Two rows of fixed-size characters suit simple status messages.	Less flexible for symbols and layout; several readings require switching between screens.
Dashboard only	Avoids a local screen and its wiring.	Cannot meet the requirement for a display at the plant, especially when the network is unavailable.

My choice: Use the SSD1306 OLED for its compact size and flexible layout. It can show current readings and short warnings, while the dashboard handles detailed history. A larger character LCD would be worth considering if the caretaker cannot comfortably read the small screen.

Select the I²C version of the OLED. I²C uses two signal wires, data and clock, and is fast enough for occasional reading updates. SPI can transfer display data faster but generally needs more signal wires; that benefit is unnecessary for this screen. Confirm the module's supply and signal levels before connecting it.

Sources: OLED displays | Character LCDs | I²C specification

6 Communication

6.1 Communication protocol – See the MQTT Research Paper for further information

Wi-Fi connects the ESP32 to the local network. A communication protocol defines how it exchanges readings and messages with software on the host computer. Watering decisions stay on the ESP32 so that this connection is not required to operate the pump.

Option	Main advantage	Main limitation
MQTT	The device sends messages to named topics through a broker, which forwards them to interested applications. Suits recurring sensor readings and events.	Requires a broker service. Message delivery settings and reconnect behaviour must be chosen deliberately.
HTTP / HTTPS	The device sends requests directly to a web service. A good option for a simple upload endpoint and familiar web integrations.	Regular updates and commands require an application-level polling or connection strategy. The device is tied more directly to that service's API.
CoAP	A compact request/response protocol designed for constrained devices; its base specification uses UDP.	Would require another integration approach for the selected backend. Its benefits are less compelling for this powered Wi-Fi prototype.

My choice: Use MQTT for measurements, watering events and any later remote settings. Publishing to topics separates the device from the program that stores readings. HTTP would also work for a small upload-only system; MQTT better matches the planned two-way message flow. MQTT acknowledgements do not prove that a database record was saved, so storage error handling is still needed.

Sources: MQTT standard | HTTP standard | CoAP standard

6.2 MQTT broker

The broker is the message-routing program between the device and the backend. It does not replace the database used for historical records.

Option	Main advantage	Main limitation
Eclipse Mosquitto	A lightweight broker that can run on the same host as the backend. Fits one device and a small number of topics.	Configuration, access control and service recovery must be managed on that host.
EMQX	Offers a management dashboard, data integrations and clustering across servers.	Adds features and operational choices that this one-device prototype does not need.
Managed MQTT service	The provider runs the broker infrastructure, reducing local broker administration.	Depends on the internet and provider terms. Charges, limits and responsibility for stored data depend on the service.

My choice: Use Mosquitto on the project host. It provides the required routing without adding a separate platform. EMQX becomes more attractive if managing many devices or broker integrations becomes part of the scope. A managed service is an alternative when maintaining a host is impractical.

Allow only the intended clients to connect and publish to the relevant topics. Use encrypted connections when messages cross untrusted networks. These are configuration responsibilities; choosing MQTT alone does not secure the system.

Sources: Mosquitto | Mosquitto authentication | EMQX

7 Backend and Database

7.1 Backend application

The backend is the program that receives device messages, checks their contents and saves them. It also handles software rules needed by the remote system; essential watering control stays on the device.

Option	Main advantage	Main limitation
Node.js	Handles incoming messages and database operations through an event-driven JavaScript program. MQTT.js and node-postgres cover the planned connections.	Asynchronous error handling must be designed carefully; a message arriving is not the same as a successful database write.
Python	Clear scripting language with asynchronous I/O support. A suitable alternative for message processing and data handling.	Would use a different set of libraries. It offers no decisive advantage for the small message-to-database service planned here.
Node-RED	Visual flows can connect device messages, processing steps and storage with less handwritten code.	Complex rules can become harder to follow across many nodes. Provides less practice in building the backend directly in code.

My choice: Use Node.js for a small, explicit message-processing service. It supports the planned MQTT and PostgreSQL connections and gives me direct experience connecting an embedded device to a backend. Python is equally viable for this workload. Node-RED is a useful alternative when rapid visual assembly matters more than implementing the service myself.

Sources: Node.js | MQTT.js | node-postgres | Python asyncio | Node-RED

7.2 Database

The database stores readings and watering events on the host computer. It must support historical queries as well as the latest values. SQL is the query language used to retrieve and organise records in relational databases.

Option	Main advantage	Main limitation
PostgreSQL	Stores readings, events and related records in structured tables. Separate applications can use controlled database accounts; Grafana has built-in support.	Requires a running database service, account configuration and backups. More administration than a single-file database.
SQLite	Stores data in a local file without a separate database server. Adequate for the modest workload of one plant.	Best suited to storage managed by an application. Using it with the chosen Grafana setup needs an additional integration choice.
InfluxDB	Designed around measurements and events over time. A natural option for sensor-history queries.	History-query limits and retention options vary by edition. Its specialised features are not essential for this small mixed set of readings and events.

My choice: Use PostgreSQL because the backend and Grafana can share structured records through established integrations. SQLite would be simpler if storage and display were combined in one application; one plant does not require PostgreSQL for capacity reasons. InfluxDB is worth revisiting if time-series processing becomes the main challenge.

The database is the persistent record; neither the local display nor the MQTT broker serves that purpose. Agree how much history to keep and maintain a recoverable backup. Readings missed during an outage need explicit local buffering if they must be recovered later.

Sources: PostgreSQL | SQLite use cases | InfluxDB 3 | Grafana PostgreSQL support

8 Dashboard and Hosting

8.1 Dashboard

The caretaker needs current readings, trends, watering history and clear warnings. Building a complete custom website is not necessary to provide these views.

Option	Main advantage	Main limitation
Grafana	Ready-made panels, time-series charts and alerting. Its PostgreSQL data source can read the stored greenhouse records.	Designed mainly for monitoring. Custom forms and remote configuration need additional design or integration.
FlowFuse Dashboard for Node-RED	Provides configurable charts, forms and controls within Node-RED flows. Useful for an interactive control interface.	Adds Node-RED to the proposed setup, or requires replacing some Node.js backend work with flows.
Custom web application	Full control over appearance, interactions and the caretaker's workflow.	Charts, accounts, data access and maintenance all require development time that could delay the core prototype.

My choice: Use Grafana for monitoring and alerts because these are the core remote tasks. Keep its database account read-only. Remote configuration is optional and would need a separate authorised write path; a monitoring dashboard should not be assumed to provide it automatically. Choose FlowFuse Dashboard or a custom interface if user interaction later becomes the main requirement.

Sources: Grafana PostgreSQL features | Grafana database permissions | FlowFuse Dashboard

8.2 Where the software runs

Mosquitto, Node.js, PostgreSQL and Grafana need a host computer. Hosting affects cost, availability and who maintains the software; watering decisions remain on the ESP32.

Option	Main advantage	Main limitation
Existing local computer	Avoids a new computer purchase. The device and dashboard can communicate on the local network without an internet connection.	Must remain powered and connected. Sleep, restarts or shared use can interrupt records and dashboard access.
Dedicated local computer	Can stay available independently of a personal laptop and keep the services in one place.	Requires purchasing hardware and providing power, storage, updates and backups.
Rented cloud server	Can be reached from outside the local network and avoids keeping a server at the plant's location.	Adds recurring charges and requires internet access for uploads. Server administration and access protection still need an owner.

My choice: Start with a suitable existing local computer. Buy a dedicated machine only if shared use or availability makes the existing one unsuitable. Cloud hosting is reasonable when off-site access justifies the cost and internet dependence. Confirm the host's capacity with the chosen software rather than selecting it by an unsupported hardware estimate.

In the selected architecture, the ESP32 sends readings through Mosquitto to Node.js. Node.js writes them to PostgreSQL, and Grafana reads the stored data for the caretaker. Local control and display operation are designed to remain available during host or network outages; verification results are reported separately.